P2573R2

`= delete("should have a reason");`

Published Proposal, 2024-03-22

This version:
https://wg21.link/P2573R2

Author:
Yihe Li

Audience:
CWG

Project:
ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Target:
C++26

Source:
github.com/Mick235711/wg21-papers/blob/main/P2573/P2573R2.bs

Issue Tracking:
GitHub Mick235711/wg21-papers

Table of Contents

6.1.2 9.5.3 Deleted definitions [dcl.fct.def.delete]

6.2 15.11 Predefined macro names [cpp.predefined]

References

Normative References

Informative References

C++ had adopted the original `static_assert` with message in C++11 by [N1720], the `[[deprecated]]` attribute with message in C++14 by [N3760], and the `[[nodiscard]]` attribute with message in C++20 by [P1301R4]. All of these introductions succeeded in introducing the ability to provide a user-defined message accompanying the generated diagnostic (warning or error), thus helping to communicate the exact intent and reasoning of the library author and generate more friendly diagnostics. This paper proposes to take a further step forward, and improve upon the other common and modern way of generating diagnostics, namely using `= delete` to delete a function, by also introducing an optional message component.

1. Revision History

1.1. R2

- Rebase onto [N4971].
- Tweaked wording.
- As per CWG guidance, changed the feature test macro to be a general one for deleted function.
- In Tokyo (2024-03), EWG sees [P2573R1] and forwarded it to CWG with the following poll:

Poll: [P2573R1] `= delete("should have a reason");` forward to CWG for inclusion in C++26.

SF	F	N	A	SA
1	11	4	0	0

Outcome: Consensus 🎉

1.2. R1

- Retargeted to C++26.
- Rebase onto [N4958], which includes [P2361R6] so remove a note for it.
- Since [P2741R3] had been adopted in Varna (2023-06), add a section for it.
- Add previous poll result for [N4186].
- In Kona (2023-11), EWGI sees D2573R1 and forwarded it to EWG with the following poll:

Poll: Given the committee's limited time, EWGI believes P2573R1 is sufficiently developed and motivated to forward to EWG (and mark for LEWG).

SF	F	N	A	SA
4	4	3	0	0

Outcome: Consensus 🎉

- Add several notes on topics discussed in EWGI meeting.

1.3. R0

- Initial revision.

§ 2. Motivation

Introduced in C++11, `= default` and `= delete` had joined `= 0` as possible alternative specification for a function body, instead of an ordinary brace-enclosed body of statements. The original motivation for deleted function declaration via `= delete` is to replace (and supersede) the C++98/03-era common practice of declaring special member functions as `private` and not define them to disable their automatic generation. However, `= delete`'s addition had gained even greater power, for it is permitted to be used for any function, not just special members. The original paper, [N2346], described the expected usage of deleted function as:

> The primary power of this approach is twofold. First, use of default language facilities can be made an error by deleting the definition of functions that they require. Second, problematic conversions can be made an error by deleting the definition for the offending conversion (or overloaded function).

Looking back at present, ten years after the introduction of deleted functions, we can confidently conclude that `= delete` had become one of the key C++11 features that greatly improved user experience on error usages of library functions and had been a success story of "Modern C++" revolution. We have seen relatively wide adoption both inside the standard library and in the wider community scope, with over 40k results for `= delete` in the [ACTCD19] database. (Though, admittedly, the feature is still mostly used for member functions, especially its original motivation, disable SMFs.)

There are several reasons we preferred deleted functions over the traditional `private`-but-not-defined ones, including better semantics (`friend` and other members are still unaccessible, turning a linker error into a compile-time error), better diagnostics (instead of cryptic "inaccessible function" errors, the user directly know that the function is deleted), and greater power (not just SMFs). As we are constantly striving to present a better and friendlier interface to the user of C++, this proposal wants to take the feature a step forward in the "better diagnostics" area: Instead of an already friendlier but still somewhat cryptic "calling deleted function" error, we directly permit the library authors to present an optional extra message that should be included in the error message, such that the user will know the exact reasoning of *why* the function is deleted, and in some cases, *which* replacement should the user heads to instead.

In other words, usage of `= delete` on a function by the library author practically means that

> The library author is saying, "I know what you're trying to do, and what you're trying to do is wrong." (Source)

and after this proposal, my hope is that such usage will mean

> The library author is saying, "I know what you're trying to do, and what you're trying to do is wrong. **However, I can tell you why I think it is wrong, and I can point you to the right thing to do.**"

The proposed syntax for this feature is (arguably) the "obvious" choice: allow an optional *string-literal* to be passed as an argument clause to `= delete`. Such a `= delete("message");` clause will be usable whenever the original `= delete;` clause is usable as a *function-body*. Thus the usage will look like this:

```
void newapi();
void oldapi() = delete("This old API is outdated and already been removed. Please use newap

template<typename T>
struct A {/* ... */};
template<typename T>
A<T> factory(const T&) {/* process lvalue */}
template<typename T>
A<T> factory(const T&&) = delete("Using rvalue to construct A may result in dangling refere

struct MoveOnly
{
    // ... (with move members defaulted or defined)
    MoveOnly(const MoveOnly&) = delete("Copy-construction is expensive; please use move cor
    MoveOnly& operator=(const MoveOnly&) = delete("Copy-assignment is expensive; please use
};
```

There is a tony table of how a non-copyable class evolves from C++98 to this proposal, thus introducing the benefits of the proposal and the user-friendliness it brings. All the examples (except the hypothetical one) are generated by x86-64 clang version 14.0.0 on Compiler Explorer. The usage client is

```
int main()
{
    NonCopyable nc;
    NonCopyable nc2 = nc;
    (void)nc2;
}
```

<table>
<tr><th>Standard</th><th>Code</th><th>Diagnostics / Comment</th></tr>
<tr><td>C++98</td><td><pre>class NonCopyable
{
public:
    // ...
    NonCopyable() {}
private:
    // copy members; no definition
    NonCopyable(const NonCopyable&);
    NonCopyable& operator=(const NonCopyable&);
};</pre></td><td><pre><source>:15:23: error: callin
private constructor of class
    NonCopyable nc2 = nc;
                      ^
<source>:8:5: note: declared
    NonCopyable(const NonCopy
    ^</pre>Average user probably don’t know what <code>friend</code>/member usage still result in link</td></tr>
<tr><td>C++11 (present)</td><td><pre>class NonCopyable
{
public:
    // ...
    NonCopyable() = default;

    // copy members
    NonCopyable(const NonCopyable&) = delete;
    NonCopyable& operator=(const NonCopyable&) = delete;
    // maybe provide move members instead
};</pre></td><td><pre><source>:16:17: error: call t
constructor of 'NonCopyable'
    NonCopyable nc2 = nc;
                ^   ~~
<source>:8:5: note: 'NonCopya
explicitly marked deleted her
    NonCopyable(const NonCopy
    ^</pre>Great improvement: we can teach "delete and everything is compile-time error. Ho understand and don’t point out what to d</td></tr>
<tr><td>This proposal</td><td><pre>class NonCopyable
{
public:
    // ...
    NonCopyable() = default;

    // copy members
    NonCopyable(const NonCopyable&)
        = delete("Since this class manages unique resources, \
copy is not supported; use move instead.");
    NonCopyable& operator=(const NonCopyable&)
        = delete("Since this class manages unique resources, \
copy is not supported; use move instead.");
    // provide move members instead
};</pre></td><td><pre><source>:16:17: error: call t
constructor of 'NonCopyable'
unique resources, copy is not
    NonCopyable nc2 = nc;
                ^   ~~
<source>:8:5: note: 'NonCopya
explicitly marked deleted her
    NonCopyable(const NonCopy
    ^</pre>With minimal change, we get a huge boo ability to explain why and what to do ins</td></tr>
</table>

There are some discussion on alternative syntax below.

§ 3. Usage Example

§ 3.1. Standard Library

This part contains some concrete examples of how current deleted functions in the standard library can benefit from the new message parameter. All examples are based on [N4971].

Note: As will be discussed below, the standard does not mandate any diagnostic text for deleted functions; a vendor has the freedom (and is encouraged by this proposal) to implement these messages as non-breaking QoI features.

```
// [unique.ptr.single.general]
namespace std {
    template<class T, class D = default_delete<T>> class unique_ptr {
    public:
        // ...
        // disable copy from lvalue
        unique_ptr(const unique_ptr&) = delete(
            "unique_ptr<T> resembles unique ownership, so copy is not supported. Use move c
        unique_ptr& operator=(const unique_ptr&) = delete(
            "unique_ptr<T> resembles unique ownership, so copy is not supported. Use move c
    }
}

// [memory.syn]
namespace std {
    // ...
    template<class T>
        constexpr T* addressof(T& r) noexcept;
    template<class T>
        const T* addressof(const T&&) = delete("Cannot take address of rvalue.");

    // ...
    template<class T, class... Args> // T is not array
        constexpr unique_ptr<T> make_unique(Args&&... args);
    template<class T> // T is U[]
        constexpr unique_ptr<T> make_unique(size_t n);
    template<class T, class... Args> // T is U[N]
        unspecified make_unique(Args&&...) = delete(
            "make_unique<U[N]>(...) is not supported; perhaps you mean make_unique<U[]>(N)
}

// [basic.string.general]
namespace std {
    template<class charT, class traits = char_traits<charT>,
             class Allocator = allocator<charT>>
    class basic_string {
    public:
        // ...
        basic_string(nullptr_t) = delete("Construct a string from a null pointer is undefin
        // ...
        basic_string& operator=(nullptr_t) = delete("Assignment from a null pointer is unde
    }
}
```

§ 3.2. Other

A hypothetical usage of new `= delete("message");` that I can foresee to be commonly adopted is to mark the old API first with `[[deprecated("reason")]]`, and after a few versions, change it to `= delete("reason");`, so that further usage of old API directly results in an error message that can explain the removal reason and also point towards the new API. Standard library vendors can also do this as a QoI feature through the freedom of zombie names.

§ 4. Design

§ 4.1. Previous Work

There have been three previous example of user-customisable error messages presented currently in the C++ standard:

- C++11 `static_assert(expr, "message")`, by [N1720].
- C++14 `[[deprecated("with reason")]]`, by [N3760].
- C++20 `[[nodiscard("with reason")]]`, by [P1301R4].

This proposal naturally fits in the same category of providing reasons/friendly messages alongside the original diagnostics and can use a lot of the existing wordings.

A previous proposal, [N4186], proposed the exactly same thing, and is received favorably by EWG in Urbana (2014-11) (see [EWG152]):

> **Poll:** Is this a problem worth solving?
>
> | SF | F | N | A | SA |
> |---|---|---|---|---|
> | 13 | 5 | 1 | 3 | 2 |

However, there is no further progress on that proposal, and thus this proposal aims to continue the work in this direction.

4.2. Syntax and Semantics

The syntax I prefer is `= delete("with reason");`, in other words, an optional argument clause after the `delete` keyword where the only argument allowed is a single *string-literal*.

The semantics will be entirely identical to regular `= delete;`, which means that the new form can be exchanged freely with the old form, with the only difference being the diagnostic message. Some additional semantics caveat is discussed in the Overriding Semantics section below.

4.3. Alternative Design and Issues

This section lists the alternative design choices and possible arguments against this proposal that have been considered.

4.3.1. Alternative Syntax

A previous proposal [P1267R0] proposed a `[[reason_not_used("reason")]]` attribute to enhance the compiler error in SFINAE failure cases. While the motivation is similar, the problem it solved is slightly different as `= delete` in general does not affect overload resolution. In its "Future Directions" section, however, a series of `[[reason_xxx]]` attributes were described, with the `[[reason_deleted("reason")]]` described will have identical semantics with what this proposal proposes. This proposal proposes:

```
void fun() = delete("my reason");
```

which in future-[P1267R0] world will be expressed as

```
[[reason_deleted("my reason")]]
void fun() = delete;
```

Personally, comparing these two syntaxes, I prefer the proposed `= delete("reason");` one as it is more concise, less noisy, has reasoning closer to the actual deletion and does not have to repeat yourself by saying first, "I delete this function" then "the reason for delete is ..." twice. An advantage I can see for [P1267R0]-like solutions is that if the committee decided to accept several `[[reason_xxx("reason")]]` family (SFINAE, `private`, `= delete`, etc.), then the attribute with the same prefix can be seen as more compatible with rest of the core language. However, since [P1267R0] is currently inactive, I will still propose the `= delete("reason");` syntax.

Of course, given that the *function-body* part of the grammar is relatively free, other (arcane?) syntax like `= default "reason";`, `= default["reason"]` can be proposed; I haven't explored these syntaxes since I think the syntax I'm proposing is the most natural one and is fitting the existing practices.

§ 4.3.2. Overriding Semantics

One of the new issues brought up by this new syntax is the overriding problem:

```
struct A
{
    virtual void fun() = delete("reason");
};
struct B : public A
{
    void fun() override = delete("different reason");
};
```

Should overriding with a different (or no) message parameter be supported? (Notice that you cannot override deleted functions with non-deleted ones or vice versa, so this is the only form.)

I believe that this should be supported for the following reason:

- Changing `= delete` message inside the library base classes should be non-breaking (non-observable).
- Existing attributes allow this (and even went further, allow an overriding function to change from having attribute to not having).

§ 4.3.3. Locales and Unevaluated Strings

I heard that there had been already some discussion in the committee for this proposal, probably in the discussion phase of [P1267R0]; the hesitancy is basically that the *string-literal*s being accepted in the structure is unrestricted, thus may raise some doubt on how to handle things like `= delete(L"Wide reason");`.

However, this is not a problem unique to this proposal. This problem also applies to the other three existing examples, and there is [P2361R6] to solve this problem in general. Therefore I do not see any reason locales of unevaluated strings should be an obstacle.

As for the problem of displaying text that cannot be represented in the execution character set, this paper follows the existing [P2246R1] changes to `static_assert` wording and use "should" to allow flexibility.

§ 4.3.4. Claim of Syntax Space

During EWGI review of this paper, some people expressed concern on the possibility of this proposal "claiming the syntax space" of possible further enhancement to `= delete`. Namely, it is possible for a future proposal to alter `= delete` the same way we altered `explicit` in C++20, to give it an optional constant argument that can express conditional `= delete`. Thus, the following code may become a possibility in future expansion:

```
void foo() = delete (sizeof(int) != 4);
```

I think this argument is ungrounded for two reasons:

- First of all, this proposal do not actually claim the full syntax space for the argument after `= delete`. Since in the C++ grammar, *string-literal*s have more priority than ordinary *expression*s, it is still possible to add the ability to put an arbitrary constant boolean expression here. The semantics will simply be to prefer this proposal's effect if a string literal is passed.
- Secondly, I feel that `= delete (<conditional-expression>)` is not a viable future direction, since you can express essentially the same meaning with `requires` today. Granted, `requires`'s semantics are different from what `= delete` does, since the former SFINAE-away the function while the latter does not affect overload resolution at all. But I still find it hard to come up with a compelling use case for conditional delete that cannot be achieved by concepts.

§ 4.4. Proposal Scope

This proposal is a pure language extension proposal with no library changes involved and is a pure addition, so no breaking changes are involved. It is intended practice that exchanging `= delete;` for `= delete("message");` will have no breakage and no user-noticeable effect except for the diagnostic messages.

There has been sustained strong push back to the idea of mandating diagnostic texts in the standard library specifications, and in my opinion, such complaints are justified. This paper does not change this by only proposing to make such "deleted with reason" ability available and nothing more.

This paper encourages vendors to apply any text they see fit for the purpose as a QoI, non-breaking feature.

§ 4.5. Future Extensions

There have been suggestions on supporting arbitrary *constant-expression*s in `static_assert` message parameter and other places so that we can generate something like

```
make_unique<int[5]>(...) is not supported; perhaps you mean make_unique<int[]>(5) instead?
```

for the invocation `make_unique<int[5]>()` (so that message come with concrete types). I do not oppose such extensions, and this proposal does not prevent supporting `= delete(constant_string)` in the future either.

During Varna (2023-06), [P2741R3] is adopted into the C++26 standard, giving `static_assert` the ability to accept a user-generated string-like object as the message parameter. However, I'm not sure if the corresponding support can be directly added to this feature, since `static_assert` is a standalone statement, and this feature is currently an attachment to a function declaration, which may lead to execution order issues. Therefore, currently I do not propose `= delete(constexpr-string-expression)` as parts of this proposal. I felt that the support for it should come together with a separate proposal to add user-generated message support to `[[deprecated]]` and `[[nodiscard]]` too.

Some people suggested considering the same message parameter for other keywords in the language, such as `final("reason")` or `explicit(<cond>, "reason")`. While such constructs might be useful, I felt that this may quickly escalate to a general facility that supports a custom message for every ill-formed program, and that is a step too far in my opinion. `= delete` is different from those two keywords in the sense that it might be used as a transitional method (to be added after the introduction of API to indicate deprecation), while the other two constructs are usually set in stone after initial API. Furthermore, the applicable scope of `final` (only on classes and virtual member functions) and `explicit` (only on constructors) are both much more narrow than `= delete` (which can be applied on any function), so I think just a controlled introduction of `= delete` should be most beneficial to users of the language without introducing too much novelty.

§ 4.6. Target Vehicle

This proposal targets C++26.

§ 4.7. Feature Test Macro

The previous works all bump the relevant attributes or feature testing macros to a new value to resemble the change. However, C++11 default and deleted functions don't have a feature test macro, so I propose to include a new language feature testing macro `__cpp_deleted_function` (name can be changed) with the usual value.

§ 5. Implementation Experience

An experimental implementation of the proposed feature is located in my Clang fork at [clang-implementation], which is capable of handling

```
void foo() = delete("Reason");
void fun() {foo();}
```

and output the following error message: (close to what I want; of course, there are other formats such as put reason on separate lines)

```
propose.cpp:2:13: error: call to deleted function 'foo': "Reason"
void fun() {foo();}
            ^~~
propose.cpp:1:6: note: candidate function has been explicitly deleted
void foo() = delete("Reason");
     ^
1 error generated.
```

The implementation is *very* incomplete (no support for constructors, no feature-test macro, only a few testing, etc.), so it is only aimed at proving that the vendors can support the proposed feature relatively easily.

§ 6. Wording

The wording below is based on [N4971].

Wording notes for CWG and editor:

- This wording factors existing `= delete;` in the *function-body* out to a new grammar token, *deleted-function-body*, to allow reusing.
- The current wording is basically a combination of `static_assert` ([dcl.pre]/12) and `[[deprecated]]` ([dcl.attr.deprecated]/n2, for error instead of warning).

§ 6.1. 9.5 Function definitions [dcl.fct.def]

§ 6.1.1. 9.5.1 In general [dcl.fct.def.general]

> Function definitions have the form

```
function-definition:
    attribute-specifier-seq_opt decl-specifier-seq_opt declarator virt-specifier-seq_opt functi
    attribute-specifier-seq_opt decl-specifier-seq_opt declarator requires-clause function-boc
function-body:
    ctor-initializer_opt compound-statement
    function-try-block
    = default ;
    = delete ;deleted-function-body
deleted-function-body:
    = delete ;
    = delete ( unevaluated-string ) ;
```

§ 6.1.2. 9.5.3 Deleted definitions [dcl.fct.def.delete]

Paragraph 1:

> A *deleted definition* of a function is a function definition whose *function-body* is ~~of the form `= delete ;`~~ a *deleted-function-body* or an explicitly-defaulted definition of the function where the function is defined as deleted. A *deleted function* is a function with a deleted definition or a function that is implicitly defined as deleted.

Paragraph 2:

> A program that refers to a deleted function implicitly or explicitly, other than to declare it, is ill-formed.

> *Recommended practice*: The resulting diagnostic message should include the text of the *unevaluated-string*, if one is supplied.

> [*Note 1*: This includes calling the function implicitly or explicitly and forming a pointer or pointer-to-member to the function. It applies even for references in expressions that are not potentially-evaluated. For an overload set, only the function selected by overload resolution is referenced. The implicit odr-use ([basic.def.odr]) of a virtual function does not, by itself, constitute a reference. The *unevaluated-string*, if present, can be used to explain the rationale for deletion and/or to suggest an alternative. — *end note*]

§ 6.2. 15.11 Predefined macro names [cpp.predefined]

In [tab:cpp.predefined.ft], insert a new row in a place that respects the current alphabetical order of the table, and substituting `20XXYYL` by the date of adoption.

Macro name	Value
`__cpp_deleted_function`	`20XXYYL`

§ References

§ Normative References

[CLANG-IMPLEMENTATION]
Yihe Li. *Mick235711's Clang Fork*. URL: https://github.com/Mick235711/llvm-project/tree/delete-with-reason

[N4186]
Matthias Kretz. *Supporting Custom Diagnostics and SFINAE*. 10 October 2014. URL: https://wg21.link/n4186

[N4971]
Thomas Köppe. *Working Draft, Programming Languages — C++*. 18 December 2023. URL: https://wg21.link/n4971

[P2573R1]
Yihe Li. *= delete("should have a reason");*. 11 November 2023. URL: https://wg21.link/p2573r1

§ Informative References

[ACTCD19]
Andrew Tomazos. *Andrew's C/C++ Token Count Dataset 2019*. URL: https://codesearch.isocpp.org

[EWG152]
Matthias Kretz. *N4186 Supporting Custom Diagnostics and SFINAE*. Open. URL: https://wg21.link/ewg152

[N1720]
R. Klarer, J. Maddock, B. Dawes, H. Hinnant. *Proposal to Add Static Assertions to the Core Language (Revision 3)*. 20 October 2004. URL: https://wg21.link/n1720

[N2346]
Lawrence Crowl. *Defaulted and Deleted Functions*. 19 July 2007. URL: https://wg21.link/n2346

[N3760]
Alberto Ganesh Barbati. *[[deprecated]] attribute*. 1 September 2013. URL: https://wg21.link/n3760

[N4958]
Thomas Köppe. *Working Draft, Programming Languages — C++*. 14 August 2023. URL: https://wg21.link/n4958

[P1267R0]
Hana Dusíková, Bryce Adelstein Lelbach. *Custom Constraint Diagnostics*. 8 October 2018. URL: https://wg21.link/p1267r0

[P1301R4]
JeanHeyd Meneide, Isabella Muerte. *[[nodiscard("should have a reason")]]*. 5 August 2019. URL: https://wg21.link/p1301r4

[P2246R1]
Aaron Ballman. *Character encoding of diagnostic text*. 15 January 2021. URL: https://wg21.link/p2246r1

[P2361R6]
Corentin Jabot, Aaron Ballman. *Unevaluated strings*. 12 February 2023. URL: https://wg21.link/p2361r6

[P2741R3]
Corentin Jabot. *user-generated static_assert messages*. 16 June 2023. URL: https://wg21.link/p2741r3